

1 Simple External Services

Simple External Services (SES) are a user-friendly interface for the application developer to develop individual services. The SES are part of the Simple Developer Interface (SDI).

1.1 Development process

1.1.1 Repository

You must first create a new service in the repository. The following attributes must be filled:

- Domain
- Function
- Service Type = **ExternalJavaService**

The following attributes of a service parameter are relevant to Simple External Services:

- Acronym
- Web Service Type
- Is result
- Input as an array
- Is Special Parameter (must always be set to Y for input parameters)
- Is mandatory
- Can *
- Transfer Empty Values To Hydra

1.1.2 Specifications for the implementation class

Package name: The class must be included in a package which starts with `de.mpdv.simpleExternalService`. Further subpackages are allowed

Class name: The class should have a name that references the function ID, e.g. *DomainFunction*

Implemented interfaces / methods: The class must implement the interface *ISimpleExternalService* and thus the method

public SesResult execute(SesRequest request, SesContext context, ISystemUtilFactory factory)

Other: The class must have a default constructor without parameters

Compilation: The Jar file *MpdvDomCoreSdiCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the Simple External Services must be stored in the following directory including package directory structure:

<JHYDRADIR>/MOC/<instance>/externalService/<scope> or

wsp_config/MOC/<instance>/externalService/<scope>

Example of a service list for the domain SampleDomain:

```
package de.mpdv.simpleExternalService;

import de.mpdv.sdi.data.SesContext;
import de.mpdv.sdi.data.SesRequest;
import de.mpdv.sdi.data.SesResult;
import de.mpdv.sdi.simpleExternalService.ISimpleExternalService;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

/**
 * Implementation for simple external service SampleDomain.list
 *
 */
public class SampleDomainList implements ISimpleExternalService
{
    /**
     * Default constructor
     */
    public SampleDomainList()
    {
        // empty
    }

    public SesResult execute(SesRequest request, SesContext context,
ISystemUtilFactory factory)
    {
        // TODO Implementation of simple external service
        return null;
    }
}
```

Directory structure in the server (instance 1, scope custom):

<JHYDRADIR>/MOC/1/externalService/custom/de/mpdv/simpleExternalService/SampleDomainList.class

1.1.3 Create mapping

To use the Simple External Service, a mapping of the function ID to the class must exist.

To this end, you must create a file *FunctionID.txt* in the folder `wsp_config/MOC/<instance>/externalServiceMapping/<scope>` or `<JHYDRADIR>/MOC/<instance>/externalServiceMapping/<scope>`. The file only includes one row that in turn includes the Fully Qualified Class Name (class name including package).

Example (Service list for domain SampleDomain with the class SampleDomainList in the package de.mpdv.simpleExternalService, instance 1, Scope custom):

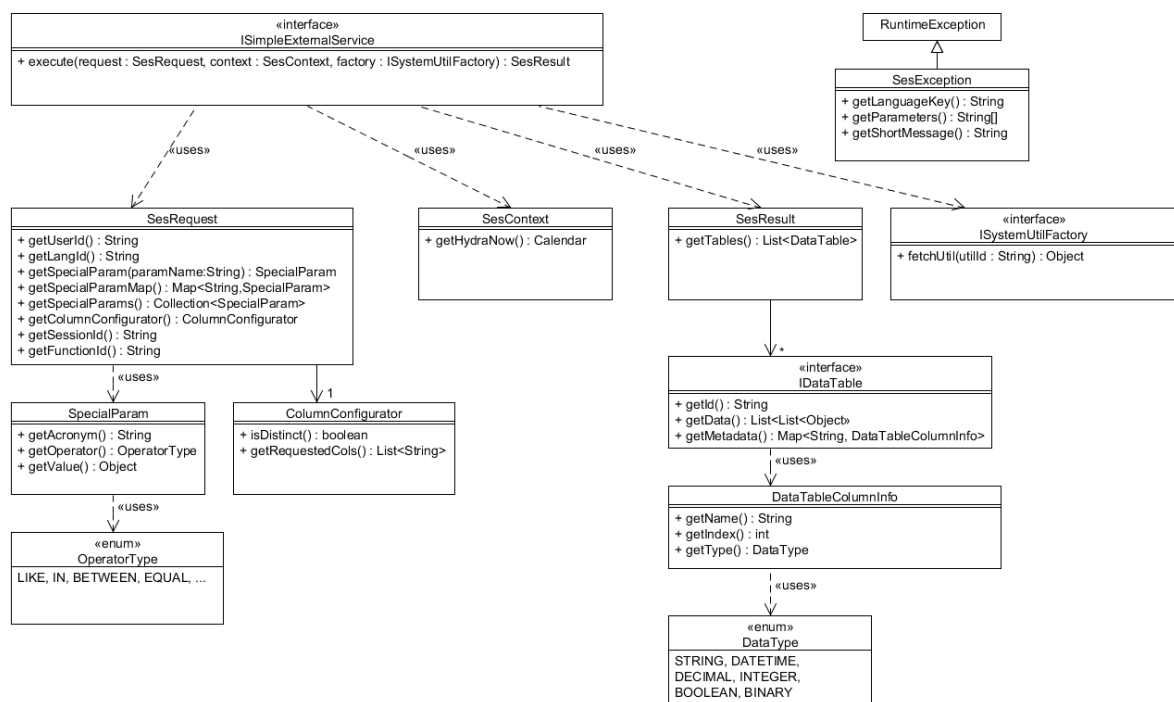
wsp_config/MOC/1/externalServiceMapping/custom/SampleDomain.list.tx: or

<JHYDRADIR>/MOC/1/externalServiceMapping/custom/SampleDomain.list.txt:

de.mpdv.simpleExternalService.SampleDomainList

1.2 Interface description

The following UML class diagram shows an overview of the relevant interface data types:



1.2.1 Interface ISimpleExternalService

This is the interface that must implement the class of a Simple External Service. The interface is included in the package *de.mpdv.sdi.simpleExternalService*.

Method	Description
execute	Method for executing the service Return SesResult - the result of the service Input

	SesRequest request - Client request. Includes, among others, SpecialParams, ColumnConfigurator SesContext context - Context object of the service. Includes additional information that might be relevant for the execution of the service. ISystemUtilFactory factory - Factory for System utils, such as logging and DB access
--	---

1.2.2 Interface ISystemUtilFactory

This is the interface that enables access to the system util functions. The interface is included in the package *de.mpdv.sdi.systemutility*.

The system utils are described in a separate section.

1.2.3 Data types

All classes described in the following are included in the package *de.mpdv.sdi.data*.

1.2.3.1 Class SesException

This class has been designed to cancel processing within a Simple External Service by an error (that can be located). It is a so-called unchecked exception (the higher-level type is RuntimeException), i.e. it does not have to be caught explicitly.

It has two constructors (with and without the optional parameter "cause")

Parameter	Type	Description
languageKey	String	Unique language key. It is transferred to the client and re-coded into the actual error message.
shortMessage	String	Brief version of the error message in

		English. It is logged in the server.
cause	Throwable	OPTIONAL The Exception that has led to throwing the SesException.
parameters	String... (0..n values possible)	Parameters for the error message, corresponding to the languageKey.

1.2.3.2 Class SesRequest

This is the data type that includes the request information. The data of this type cannot be changed.

Method	Description
getUserId	Provides the UserId of the user logged on to the client Return type: String
getLangId	Returns the language abbreviation of the client. Return type: String
getSpecialParam	Provides the special parameter for a transferred parameter name Return type: String Input: String paramName – the parameter name
getSpecialParamMap	Provides a map including all special parameters. The parameter name is the key. Return type: Map<String, SpecialParam>
getSpecialParams	Provides a collection including all special parameters. Return type: Collection<SpecialParam>

getColumnConfigurator	Provides the column configurator for the request. It informs on the columns that have been requested from the client. Return type: ColumnConfigurator
getFunctionId	Provides the Function ID of the requested service. Return type: String
getSessionId	Provides the session ID of the current client session. Return type: String

1.2.3.3 Class SpecialParam

This is the data type that includes information on a special parameter. The data of this type cannot be changed.

Method	Description
getAcronym	Provides the acronym of this special parameter Return type: String
getOperator	Provides the operator of this special parameter Return type: OperatorType (enum)
getValue	Sends the value of this special parameter Return type: Object The actual data type depends on the parameter type. These types are possible: - String - Integer - BigDecimal - Boolean - Byte[] - Calendar - String[]

	- Integer[] - BigDecimal[] - Boolean[] - Calendar[]
--	--

1.2.3.4 Class OperatorType

It is an **enum** representing different operators. The following values are possible:

Value	Description
LIKE	Like a specific value
EQUAL	Equal to a specific value
BETWEEN	Between two specific values
IN	Included in a quantity of specific values
NOT_EQUAL	Unequal to a specific value
LIKE_OR_NULL	Like a specific value or null
EQUAL_OR_NULL	Equal to a specific value or null
BETWEEN_OR_NULL	Between two specific values or null
IN_OR_NULL	Included in a quantity of specific values or null
NOT_EQUAL_OR_NULL	Unequal to a specific value or null
GT	Greater than a specific value
LT	Less than a specific value
GTE	Greater than or equal to a specific value
LTE	Less than or equal to a specific value
GT_OR_NULL	Greater than a specific value or null
LT_OR_NULL	Less than a specific value or null
GTE_OR_NULL	Greater than or equal to a specific value or null
LTE_OR_NULL	Less than or equal to a specific value or null

1.2.3.5 Class ColumnConfigurator

This is the data type that includes information on the column configurator of a request. The column configurator informs about the columns that have been requested by the client.

Method	Description
getRequestedCols	Provides a list of column names requested by the client. Return type: List<String>
isDistinct	Specifies whether or not double values may be included in the result DataTable of the service.

	Return type: Boolean
--	-----------------------------

1.2.3.6 Class SesContext

This is the data type that includes additional context information. The data of this type cannot be changed.

Method	Description
getHydraNow	Provides the NOW time for the current service call (HYDRA Now) Return type: Calendar

1.2.3.7 Class SesResult

This is the data type that includes result information on a Simple External Service. The Builder class SesResultBuilder (see section "Class SesResultBuilder") is available for the generation of a SesResult object.

The constructor has the following parameters:

Parameter	Type	Description
tables	List<IDataTable>	List of all DataTables returned to the client.

1.2.3.8 Interface IDataTable

This is the data type that includes a data table. To create a DataTable object, the utility "DataTableBuilder" is available (see section "Interface ISystemUtilFactory").

The constructor has the following parameters:

Parameter	Type	Description
id	String	The unique ID of the data table
data	List<List<Object>>	The content of the data table. An indexed list of data rows. A data row, in turn, is an indexed list of column values. The actual data type of a column value depends on the definition of meta

		data (see below).
metadata	Map<String, DataColumnColumnInfo>	The definition of columns. Column information is mapped including column names as the key.

1.2.3.9 Class DataColumnColumnInfo

This is the data type that includes information on a column. DataColumnColumnInfo objects are generated implicitly using the utility "DataTableBuilder" (see section "Interface ISystemUtilFactory").

The constructor has the following parameters:

Parameter	Type	Description
Name	String	Column name
index	int	Column index (matches the index within the list<Object>object of a row)
Type	DataType (enum)	The data type of the column content

1.2.3.10 Class DataType

It is an **enum** representing different data types. The following values are possible:

Value	Corresponding Java data type
STRING	java.lang.String
DATETIME	java.util.Calendar
DECIMAL	java.math.BigDecimal
INTEGER	java.lang.Integer
BOOLEAN	java.lang.Boolean
BINARY	byte[]

1.2.3.11 Class SdiException

You use this class to cancel the processing in SDI with an error (that can be located). It is a so-called unchecked exception (the higher-level type is RuntimeException), i.e. it does not have to be caught explicitly.

Description of the interface, see SesException.

1.2.4 Builder

1.2.4.1 Class DataTableBuilder



This class is outdated since SdiCompileLib version 1.12! Do not continue to use! Use instead the utility "DataTableBuilder" of type IDDataTableBuilder of the ISystemUtilFactory. In the class DataTableBuilder, the monitoring of memory usage is not included that should prevent OutOfMemoryExceptions!

Element	Description
Constructor	IMPORTANT: outdated! Use instead the utility DataTableBuilder of the ISystemUtilFactory. Generates the builder Result: DataTableBuilder Input:
Method setDataTableId	Sets the ID of the DataTable Result: DataTableBuilder Input: String dataTableId
Method addCol	Adds a column to the table (meta info). Result: DataTableBuilder Input: String colName – column name

	DataType columnType – column type (enum)
Method addRow	Adds a row to the table and sets the transferred column values. Result: DataTableBuilder Input: Object... values (0..n values) – The values to be set
Method addRow	Adds an (empty) row to the table. You can then set the column values one after the other using the method value. Result: DataTableBuilder Input:
Method value	Adds the value of the next column to the current table row. The number of defined columns specifies how often this method can be called for each row. Result: DataTableBuilder Input: Object value – The value to be set
Method build	Return of the ready-designed DatatTable object. Result: IDataTable Input:

1.2.4.2 Class SesResultBuilder

This is a builder providing support with the creation of a SesResult object. This class is included in the package *de.mpdv.sdi.data*.

Element	Description
Constructor	Generates the builder Result: SesResultBuilder Input:
Method addDataTable	Adds a DataTable. Result: SesResultBuilder Input: IDataTable dataTable
Method build	Return of the ready-designed SesResult object Result: SesResult Input:

1.2.4.3 Class SesExceptionBuilder

This is a builder that you can use to create a SesException with more than one language key (more than one message). This class is included in the package *de.mpdv.sdi.data*.

Element	Description
Constructor	Generates the builder Result:

	SesExceptionHandler Input:
Method cause	Provides the exception that has led to throwing the SesExceptionHandler (optional). Result: SesExceptionHandler Input: Throwable pCause
Method languageKey	Adds a new language key. Result: SesExceptionHandler Input: String langKey
Method shortMessage	Provides the short description of the error for the current language key. Result: SesExceptionHandler Input: String shortMessage Exception: IllegalStateException if no language key has been added yet.
Method parameters	Provides the parameters for the current language key. Result:

	SesExceptionHandler Input: String... parameters Exception: IllegalStateException if no language key has been added yet.
Method build	Creates the Exception Result: SesExceptionHandler Input: - Exception: IllegalStateException if not a single language key has been added.

1.2.4.4 Class SdiExceptionHandler

This is a builder that you can use to create a SdiExceptionHandler with more than one language key (more than one message). This class is included in the package *de.mpdv.sdi.data*.

Interface *sie* SesExceptionHandler (return is SdiExceptionHandler / SdiExceptionHandler instead of SesExceptionHandler / SesExceptionHandler)

1.3 Best Practices

1.3.1 Exception Handling in Simple External Services

Relevant areas in Simple External Services should be surrounded by try Catch.

In this context, it is important not to execute a "Catch all", i.e. NOT

```
catch (final IOException e)
```

OR NOT

catch (final Throwable t)

but only to catch the exceptions that may actually occur within try/catch (so-called Checked Exceptions that Eclipse shows for each error even if they are not caught).



This is important to make sure that exceptions from the system's basis provided with a specific language key are actually sent to the client and are not caught in the Simple External Service and replaced by another message.

In general, language keys should always be used for exceptions in order for the client to show localized error messages.

You must store the texts for the language key in the client and the client then shows the text instead of the language key. You can implement that the text of the language key is identified with respect to the language used.

Furthermore, it is important to forward the exception that has been caught.

Example of a general error:

```
throw new SesException("lkErrorAtExternalServiceExecution",
    "Error at execution of simple external Service.",
    e);
```

1.3.2 Creating a DataTable

To create a DataTable as the result of the Simple External Service, you can use the utility "DataTableBuilder" of the ISystemUtilFactory of type IDDataTableBuilder. Using the builder, you first set the meta information on the columns.

Example:

```
IDDataTableBuilder dataTableBuilder = factory.fetchUtil("DataTableBuilder");
dataTableBuilder.setDataTableId("returnTable1");
dataTableBuilder.addCol("retColumnString2", DataType.STRING);
dataTableBuilder.addCol("retColumnInt1", DataType.INTEGER);
```

Then the data is added. There are two options.

Example of option 1:

```
dataTableBuilder.addRow();
dataTableBuilder.value("myString1");
dataTableBuilder.value(Integer.valueOf(12));
dataTableBuilder.addRow();
dataTableBuilder.value("myString2");
```

```
dataTableBuilder.value(Integer.valueOf(22));
```

This option is best if the number of columns is dynamic, i.e. depends on the ColumnConfigurator.

Example of option 2:

```
dataTableBuilder.addRow("myString1", Integer.valueOf(12));  
dataTableBuilder.addRow("myString2", Integer.valueOf(22));
```

This option is best if the number of columns is fixed.